Tema 1
Punteros y memoria dinámica

Gabriel Navarro



Metodología de la Programación

Grado en Ingeniería Informática

Índice

- Punteros
  - Definición y ejemplos
  - Punteros y vectores
  - Punteros y matrices
  - Punteros y cadenas de caracteres
  - Punteros y estructuras
- Memoria dinámica
  - Motivación
  - Esquema de uso
  - Operador new y delete
  - Operador new [] y delete []
  - Reserva de matrices dinámicas
  - Listas enlazadas

El operador de dirección &

El **operador unario &**, precediendo al identificador de una variable o constante, devuelve su dirección de memoria.

Ejemplo

```
#include <iostream>
using namespace std;
int main() {
  int a= 3;
  double b= 7.8;

  cout<<"Dirección de memoria de a: "<< &a << ", con valor " <<a ;
  cout<<"Dirección de memoria de b: "<< &b << ", con valor " <<b ;
  return  0;
}
```

```
Dirección de memoria de a: 0x23ff74, con valor 3
Dirección de memoria de b: 0x23ff78, con valor 7.8
```

El operador de dirección &

¡¡¡CUIDADO!!!

¿Qué diferencia existe en el uso de **&** en los siguientes casos?

- void **PasoPorReferencia(**int &a**)**
- int a=4, b=6, c; c= a&b;
- bool a=true, b=true, c; c= a&&b;
- bool a=true; **cout**<<&a;

El operador de dirección &

¡¡¡CUIDADO!!!

¿Qué diferencia existe en el uso de **&** en los siguientes casos?

- void **PasoPorReferencia(**int &a**)** *// Paso por referencia*
- int a=4, b=6, c; c= a&b; *// Op. AND a nivel de bits*
- bool a=true, b=true, c; c= a&&b; *// Op. AND lógico*
- bool a=true; **cout**<<&a; *// Op. de dirección*

El operador & puede aplicarse sobre cualquier dato del programa almacenado en memoria.

Ejemplos

```
int v[100]; const double b;
cout<<&v<<' '<<&(v[0])<<' '<<&b;
```

En el ejemplo, ¿Qué diferencia hay entre &v, v y &(v[0])?

El Tipo de Dato *puntero*

Definición

Un puntero es un tipo de dato cuyos valores son direcciones de memoria que contienen valores de otro tipo de dato base.

Declaración

```
TipoDeDatoBase  *identificador;
```

identificador es una variable cuyos valores son direcciones de memoria que contienen datos de tipo *TipoDeDatoBase*

Ejemplo: Declaración de puntero a entero y a double

```
int *p_entero;
double *p_real;
```

El Tipo de Dato *puntero*

A una variable de tipo puntero se le puede asignar la dirección de memoria de cualquier dato de su tipo base.

Ejemplos:

```
char  a[10]= "Hola", b= 'g', *pchar;

pchar= a; // Dirección de la primera componente del vector
cout<<pchar<<" y "<<&(a[1]);
cout<<pchar[0]<<" y "<<a[0] // Muestra "o y H"

pchar= &b; // Dirección de b
```

El operador de indirección *

El **operador unario** *, precediendo al identificador de una variable o constante de tipo puntero, devuelve el valor del dato al que referencia.

Ejemplo

```
int main() {
   int x=2; // variable tipo entero
   int *p=&x; // puntero a variable tipo entero
   cout << *p << endl;  // valor de 'x'
   x=3;
   cout << *p << endl;  // valor de 'x'
   return 0;
}
```

```
2
3
```

El operador de indirección *

¡¡¡CUIDADO!!!

¿Qué diferencia existe en el uso de en los siguientes casos?

- int a= 4, b= 5; **cout**<<a*b
- int *c= &a;
- **cout**<<*c;

El operador de indirección *

¡¡¡CUIDADO!!!

¿Qué diferencia existe en el uso de en los siguientes casos?

- int a= 4, b= 5; **cout**<<a*b *// Op. de multiplicación*
- int *c= &a; *// Declaración de puntero*
- **cout**<<*c; *// Op. de indirección*

El operador de indirección *

Ejemplo: Combinando & y *

```
int  a= 4, *p;
p= &a;
*p= 7;
cout<<"El valor de a ahora es "<<a;
cout<<&p<<" es la dirección de memoria de p";
cout<<p<<" es la dirección de memoria de a (el valor de p)";
cout<<*p<<" es el valor del dato al que referencia p (a)";
cout<<&a<<" es la dirección de memoria de a";
cout<<a<<" es el valor de a";

cout<<*(&a)<< "Es el dato que hay en la dirección de memoria de a
(a)";
cout<<*(&(*(&a)))<< "Es el dato que hay en la dirección de memoria
del dato que hay en la dirección de memoria de a (a)";
```

MORALEJA

Los operadores **&** y ***** se anulan mutuamente cuando están seguidos.

Punteros y vectores

Los punteros y vectores están estrechamente vinculados.

Relación entre vectores y punteros

Al declarar un vector tipo identificador`[num_elem]`

1. Se reserva memoria para almacenar `num_elem` elementos de tipo tipo .
2. Se crea un puntero CONSTANTE llamado `identificador` que apunta a la primera posición de la memoria reservada.

Por tanto, el identificador de un vector, es un puntero CONSTANTE a la dirección de memoria que contiene el primer elemento. Es decir, `v` es igual a `&(v[0])`

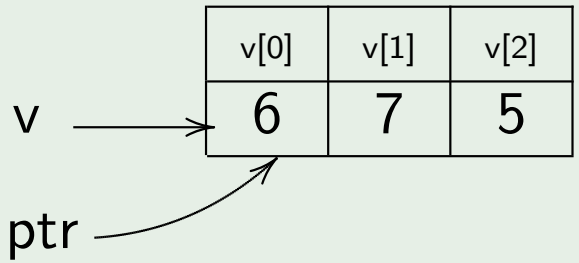
Equivalencia clave

`v=&v[0]`

Punteros y vectores

Por tanto, podemos hacer...

```
int v[3];
int *ptr;
ptr = &(v[0]); // equivalentemente, ptr=v
```



`v[0]=6` es equivalente a `*v=6` y a `*(&v[0])=6`

A los punteros se les pueden poner subindices y utilizarlos **como si fuesen vectores**, `v[i]` es equivalente a `ptr[i]`

Aritmética de punteros

Los punteros son direcciones de memoria, pero pueden verse representados como enteros.

Literales puntero

C++ no permite la asignación de literales *direcciones de memoria*, salvo la excepción de la dirección nula NULL (valor entero 0).

Ejemplo:

```
double *p= 0; // Asignación de la dirección nula a p
```

*int *p= 0;* y *int *p=* **NULL** *;* son equivalentes. **NULL** es un valor constante declarado en la biblioteca **iostream** con el valor entero de la dirección nula 0.

Aritmética de punteros

C++ permite realizar con punteros algunas operaciones aritméticas y de comparación propias de números enteros:

Operaciones permitidas sobre punteros

+,-, ++, −−, ==, !=, >, <, <=, >=

Ejemplo

```
bool  v[10], *pi, *pd;
pi= &(v[3]); pd= &(v[9]);
cout<<"Componentes la 4a. y la 10a.: "<<pd-pi+1;
cout<<*(pi+1) << ", el valor de la siguiente componente de pi";
cout<<pi[1] << ", el valor de la siguiente componente de pi";
```

Aritmética de punteros

Ejemplo: Recorrer un vector con punteros

```
int  v[10]= {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
int  *pi, *pd= &(v[9]);

for (pi= &(v[0]); pi != pd; pi++)
    cout<<"Componente en "<<pi<<" vale "<<*pi<<'\n';
```

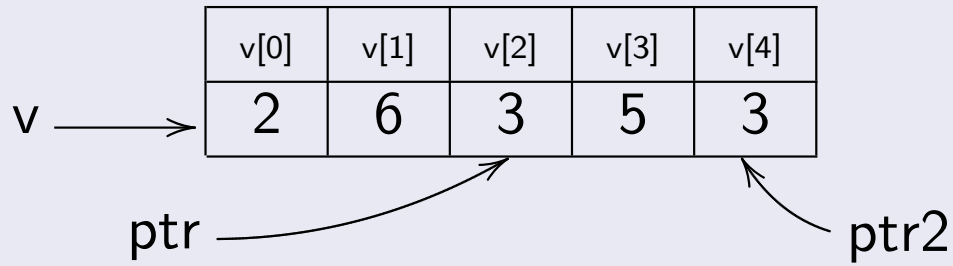
Ejemplo 2: Recorrer un vector con punteros

```
int  v[10]= {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
int  *pi= v, i;

for (i= 0; ij10; i++) {
    cout<<"Componente en "<<i<<" vale "<<*(pi+i)<<'\n';
    cout<<"Componente en "<<i<<" vale "<<pi[i]<<'\n';
}
```

Aritmética de punteros

Operadores con punteros



- Operaciones entre punteros que referencian a objetos del mismo vector
  - `ptr==ptr2;` // es falsa, apuntan a diferentes direcciones // pero *ptr==*ptr2 es verdadera
  - `ptr!=ptr2;` // es verdadera
  - `ptr < ptr2; ptr <= ptr2` // son verdaderas
  - `ptr > ptr2; ptr >= ptr2` // son falsas
  - Como `ptr2 > ptr` es verdadera, la expresión `ptr2-ptr` es valida y devuelve el número de elementos entre `ptr` y `ptr2`

Aritmética de punteros

Ejemplo: las matrices se representan en memoria por filas y de forma secuencial

Las matrices se representan en memoria por filas

```
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
  const int FILAS=2, COLUMNAS=3, ALTURAS=2;

  int matriz2D[FILAS][COLUMNAS]={{1,2,3},{4,5,6}};
  int matriz3D[FILAS][COLUMNAS][ALTURAS]=
                    {{{1,2},{3,4},{5,6}},
                    {{7,8},{9,10},{11,12}}};

  int *puntero;// puntero de acceso a cada casilla
```

Aritmética de punteros

Las matrices se representan en memoria por filas

```
puntero=&(matriz2D[0][0]);// apuntamos a primera casilla
for (i=0;i<FILAS*COLUMNAS;i++,puntero++)
   cout << setw(3) << *(puntero);

cout << endl;

puntero=&(matriz3D[0][0][0]);// ap. a primera casilla
for (i=0;i<FILAS*COLUMNAS*ALTURAS;i++,puntero++)
   cout << setw(3) << *(puntero);

return 0;}
```

```
1 2 3 4 5 6
1 2 3 4 5 6 7 8 9 10 11 12
```

Autoevaluación

Ejercicios de Autoevaluación

- Modificar el algoritmo de búsqueda secuencial del tema de vectores para que el bucle se recorra con el uso de un puntero.
- Modificar el algoritmo de ordenación por selección del tema de vectores para que todas las variables utilizadas (menos los parámetros a la función) sean punteros.

Punteros y matrices

Aunque la disposición en memoria de una matriz sea por filas (secuencialmente), el identificador de una matriz NO es un puntero a la casilla `[0][0]`

Ejemplo

```
.....
int a[4][5];
int *q;
q=a;
.....
```

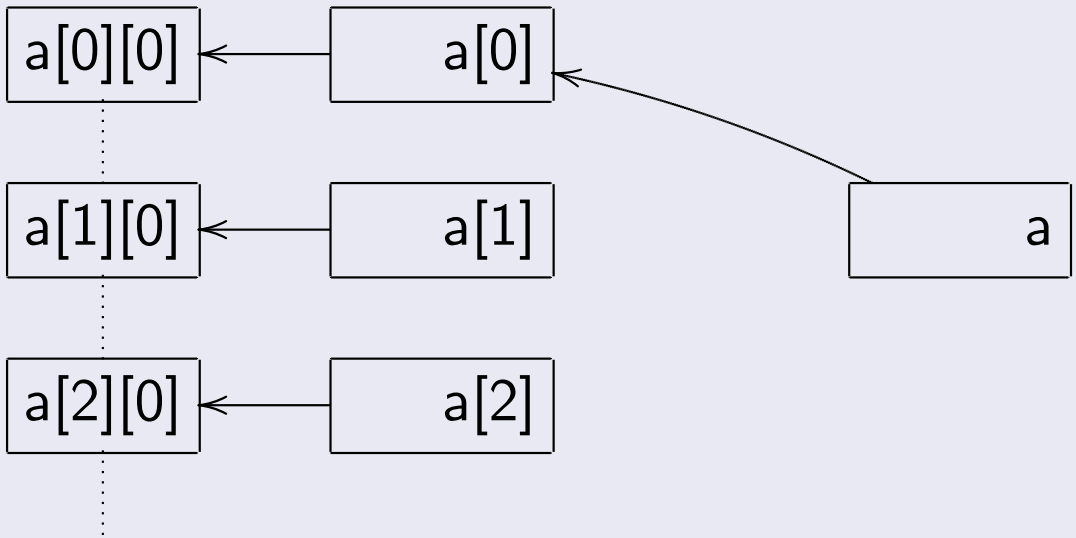
Error: cannot convert 'int[4][5]' to 'int*' in assignment

Punteros y matrices

Funciona como un **puntero a un puntero a la casilla** `[0][0]`

Gráficamente

```
int a[3][5];
```



Equivalencia

```
matriz[a][b]=*(*(matriz+a)+b)
```

Punteros y matrices

No es posible la siguiente asignación

```
int a[4][5];
int **q;
q=a;
```

Error: cannot convert 'int[4][5]' to 'int**' in assignment

La asignación debe ser como un vector de punteros

```
int a[4][5];
int *q[4];
for (int i=0;i<4;i++)
   q[i]=a[i] // sin problemas
```

Punteros y matrices

Ejemplo de vector de punteros

```
int main() {
  const int TOPE=4;
  int *v[TOPE];// Vector de TOPE punteros a enteros
  int i, a = 5, b = 10;

  v[0] = v[1] = &a; // v[0] y v[1] apuntan a 'a'
  v[2] = v[3] = &b; // v[2] y v[3] apuntan a 'b'

  for (i = 0; i < TOPE; i++) // Mostrar el contenido de los
     cout << *(v[i]) << " "; // objetos referenciados por 'v[i]'

  return 0;} //
```

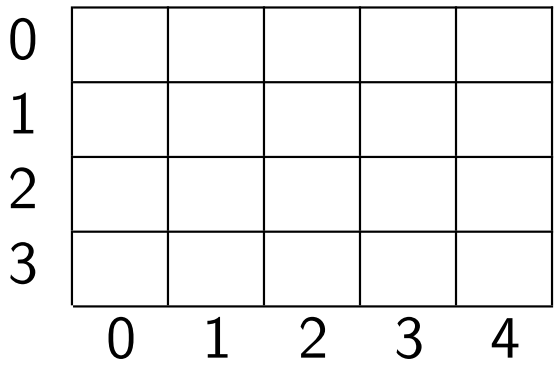
---

5 5 10 10

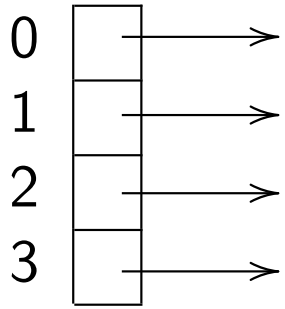
Punteros y matrices

Tenemos dos posibles estructuras de datos:

- `int a[4][5];` una matriz de 4 filas y 5 columnas
- `int *b[4];` un vector de cuatro punteros



Representación en memoria de a



Representación en memoria de b

Punteros y matrices

Comentarios

- Las expresiones `a[3][4]` y `b[3][4]` son referencias sintácticamente válidas a un entero
- Para `b[3][4]`, al no haber sido reservada la zona de memoria:
  - La lectura nos devolverá valores "basura" (también `a[3][4]`)
  - La escritura da error en tiempo de ejecución
- La iniciación de las zonas de memoria referenciadas por `b[i]` debe hacerse de forma explícita
  - De forma estática, con un vector de punteros o una matriz
  - Utilizando funciones de asignación de memoria dinámica

Diferencia

Las "filas" de un vector de punteros no tienen porqué estar almacenadas en memoria de forma secuencial

Ejemplo

```
int main() {
  int a[2][4]={{1,2,3,4}};
  int *b[2];
  int v[]={-1,-1,-1,-1};
  b[0]=&(a[0][0]);  // o bien, b[0]=a[0]
  b[1]=v;
  int *pm=b[0];
  for (int j=0;j<5;j++,pm++)
     cout << *pm << " ";
return 0;}
```

---

1 2 3 4 0 // ¡el último no es -1!

Punteros y cadenas de caracteres

Cadenas de caracteres

Son vectores de caracteres cuyo número de componentes útiles está delimitado por el caracter '\0' (delimitador de fin de cadena). Al ser un vector de caracteres, una cadena de caracteres es un tipo de dato char *

¡¡¡CUIDADO!!!

No todo vector de caracteres es una cadena, se necesita finalizar con el carácter '\0':

- Iniciando de forma adecuada la cadena
- Utilizando funciones de la libreria `cstring`
- Utilizando funciones diseñadas por el programador

Punteros y cadenas de caracteres

Formas de inicializar una cadena de caracteres

- Especificando el tamaño
  ```
  char cadena1[5]={'H','o','l','a','\0'};
  ```
- Sin especificar el tamaño
  ```
  char cadena2[]={'H','o','l','a','\0'};
  ```
- Como un literal delimitado por dobles comillas
  ```
  char cadena4[]="Hola"; // no constante
  char *cadena5="Hola"; // ¡cuidado! constante
  ```

¡¡¡CUIDADO!!!

Al declarar cadenas de caracteres, tenemos que tener siempre en cuenta que hay que reservar un caracter de más para el '\0'.
Ejemplo: `char c[10]; // Cadena de 9 caracteres max.`

Punteros y cadenas de caracteres

Operador de escritura de cadenas

El operador << está sobrecargado para cadenas de caracteres clásicas (= también muestra por pantalla cadenas de caracteres)

Ejemplo del operador <<

```
int main() {
  char saludo0[5] = { 'H', 'o', 'l' , 'a' , '\0'};
  char saludo1[] = { 'H', 'o', 'l' , 'a' , '\0'};
  char *saludo2 = "Hola";
  cout << saludo0 << " "
       << saludo1 << " " << saludo2;
  cout << &saludo2;// no muestra dirección!
}
```

Hola Hola Hola Hola

Punteros y cadenas de caracteres

Función de lectura de cadenas

```
istream & getline (char *p, int n);
```

- Lee, como mucho, $n-1$ caracteres de la entrada estandar
- Termina antes si se introduce '\n' (ENTER)
- Los caracteres leidos los copia (por orden) en `p`, salvo el '\n', que lo sustituye por '\0'
- La memoria referenciada por `p` debe estar reservada y ser suficiente

¡¡¡CUIDADO!!!

También se puede usar `cin >> cadena` pero finaliza al encontrar un espacio!

Punteros y cadenas de caracteres

Ejemplo de lectura de cadenas

```
#include <iostream>
using namespace std;
int main() {

  const int  TAM = 80;
  char  cad[TAM];

  cout << "Introducir cadena: ";

  cin.getline (cad, TAM);

  cout << "Cadena: " << cad << endl;

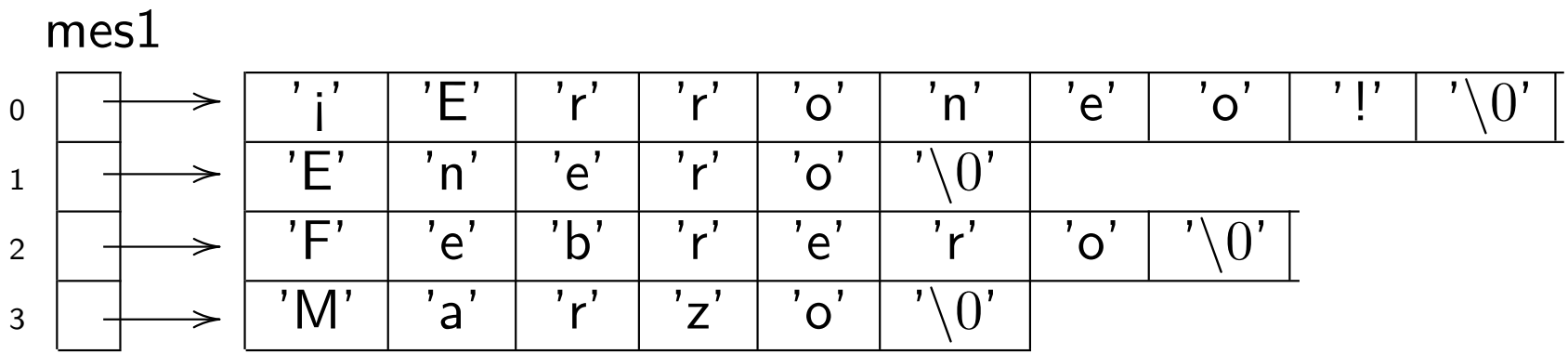
 return 0;
}
```

Punteros y cadenas de caracteres

Tenemos dos posibles estructuras de datos para un conjunto de cadenas de caracteres

- Un vector de punteros a carácter

```
char *mes1[]={"!Erroneo!","Enero","Febrero","Marzo"};
```



Se les llama ragged arrays o vectores "a jirones"

Punteros y cadenas de caracteres

Tenemos dos posibles estructuras de datos para un conjunto de cadenas de caracteres

- Una matriz bidimensional de caracteres

```
char *mes2[][10]={"!Erroneo!","Enero","Febrero","Marzo"};
```

mes2

0	'¡'	'E'	'r'	'r'	'o'	'n'	'e'	'o'	'!'	'\0'
1	'E'	'n'	'e'	'r'	'o'	'\0'				
2	'F'	'e'	'b'	'r'	'e'	'r'	'o'	'\0'		
3	'M'	'a'	'r'	'z'	'o'	'\0'				
	0	1	2	3	4	5	6	7	8	9

Punteros y cadenas de caracteres

Comentarios al respecto

- La ventaja de utilizar vectores de punteros frente a matrices dimensionadas es que **cada línea puede tener un número arbitrario de columnas**, optimizándose el espacio en memoria
- Un vector de punteros (convenientemente inicializado) se comporta como una matriz bidimensional, ya que se puede acceder a los elementos de esta estructura de datos mediante índices, como si de una matriz se tratara.

Diferencia clave

En un vector de punteros las "filas" no tienen porqué estar almacenadas de forma secuencial en memoria

Punteros y estructuras

Al igual que con los tipos de dato base, podemos crear punteros a estructuras

Ejemplo:

```
Punto2D  punto;
Punto2D  *punteroapunto= &punto;
```

La variable punteroapunto es una dirección de memoria a una zona que contiene un Punto2D.

*punteroapunto es la propia estructura a la que apunta.
(*punteroapunto).x es el campo x de la propia estructura a la que apunta.

El operador $->$

C++ simplifica el acceso a campos de estructuras referenciadas con el operador $->$.

Ejemplo: **punteroapunto**$->$**x** equivale a (*punteroapunto).x

Punteros y estructuras

Ejemplo:

```
Punto2D  punto= {5.0, 2.3}; // Punto (5, 2.3)
Punto2D  *ppunto= &punto;

cout<<(*ppunto).x << " equivale a " ppunto->x;
```

El operador -> es muy usual en C++. Se utilizará con mayor frecuencia en los siguientes temas y en la asignatura *Estructuras de Datos*.

Punteros a punteros

C++ permite la definición de datos de tipo *dirección de memoria que apunta a un dato que es una dirección de memoria que apunta a un dato* de un tipo base.

Ejemplo: Punteros dobles y triples

```
int v[10]= {0};
int  *pentero;
int  **pv;
int  ***ppv;

pentero= &(v[4]);
pv= &v;
ppv= &pv;
```

Los punteros múltiples son muy útiles en programación.

Memoria Dinámica

Estructura de la memoria

Recordemos que al ejecutar un programa, se crea un entorno con la siguiente estructura:



Segmento de código

- Almacena el código del programa. Por ejemplo, el código de todas las funciones
- Los punteros a funciones apuntan a esta parte de la memoria
- Suele ser memoria de sólo lectura

Estructura de la memoria

Segmento de datos estáticos

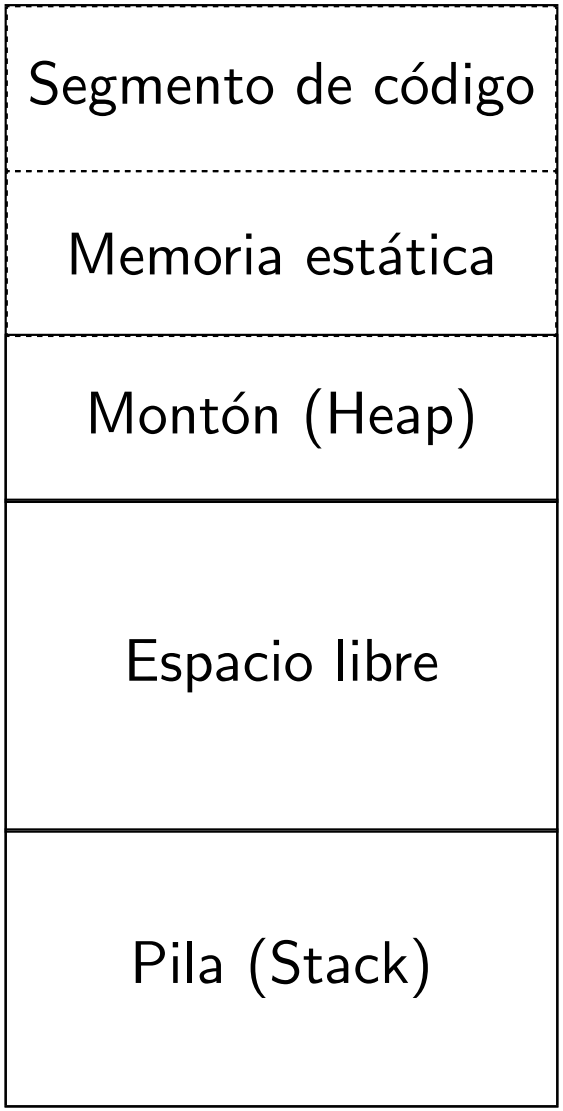
- Almacena los objetos cuya vida se extiende durante todo el programa. Ejemplo: variables o constante globales o de tipo static
- Reserva antes de la ejecución del programa
- Tiene un tamaño fijo
- No requiere gestión durante la ejecución, el S.O. se encarga de la reserva, recuperación y reutilización
- Se divide en dos partes:
  1. Datos estáticos no iniciados
  2. Datos estáticos iniciados



Estructura de la memoria

La Pila (Stack)

- Memoria que el programa utiliza para guardar los datos sobre la ejecución del programa (llamada a funciones, dirección de instrucción de retorno, etc..)
- Es una zona de datos de tipo LIFO (last in first out)
- La reserva y liberación de la memoria la realiza el S.O. de forma automática durante la ejecución del programa
- No tiene un tamaño fijo y puede llegar a ocupar parte del montón
- Las variables locales no son variables estáticas. Son un tipo de variables dinámicas (**variables automáticas**)



Estructura de la memoria

El Montón (Heap)

- Memoria que el programador utiliza para reservar nuevos datos a lo largo de la ejecución
- Las asignaciones de memoria son más lentas que en la pila
- La destrucción de los objetos dependen del programador
- Ventaja: **mayor control** sobre el uso de la memoria
- Inconvenientes: **"perdidas" de memoria** al no destruir los objetos, **fragmentación** de la memoria

A los objetos almacenados en el montón se les llama **objetos dinámicos**



Motivación: Uso del montón

Curso 2007/2008, Número alumnos de MP2: 17

Programa para calcular la media de las notas

```
int main() {
  double notas[17], media=0;
  for(int i=0; i<17 ;i++) {
     cin >> notas[i];
     media=media+notas[i];
  }
  media=media/17;
  return 0;}
```

Curso 2008/2009, Alumnos de MP2: 22. No sirve
Curso 2009/2010, Alumnos de MP2: 10. Desperdicia memoria

El programa es poco general

Motivación: Uso del montón

Problema: conocer el número en **tiempo de ejecución**

Un apaño con memoria estática

```
int main() {
  int n;
  cout << "Numero de alumnos: " << endl;
  cin >> n;

  double notas[n], media=0;
  for(int i=0; i<n ;i++) {
     cin >> notas[i];
     media=media+notas[i];
  }
  media=media/n;
  return 0;}
```

Funciona, pero es **poco conveniente**

Motivación: Uso del montón

Otro problema: Que no haya suficiente memoria

Supongamos que sólo podemos almacenar 300 enteros

```
// Calculo media de edad de 200 hombres y 200 mujeres
int main() {
 int hombres[200], media=0;
 int mujeres[200]; // No hay sufiente memoria
 for(int i=0; i<200 ;i++) {
    cin >> hombres[i];
    media=media+hombres[i];}
 for(int j=0; j<200 ;j++) {
    cin >> mujeres[j];
    media=media+mujeres[j];}
 media=media/400;
 return 0;}
```

Motivación: Uso del montón

Una manera de salvar la situación

Supongamos que sólo podemos almacenar 300 enteros

```
// Calculo media de edad de 200 hombres y 200 mujeres
int main() {
 int hombres[200], media=0;
 for(int i=0; i<200 ;i++) {
    cin >> hombres[i];
    media=media+hombres[i];}
 for(int j=0; j<200 ;j++) {
    cin >> hombres[j];
    media=media+hombres[j];}
 media=media/400;
 return 0;}
```

Es un poco chapucero

Motivación: Uso del montón

Último ejemplo: redimensionar un vector

```
int main() {
  int v[30];
  for (int i=0;i<30;i++)
     cin >> v[i];
  cout << media (v,30); // función que calcula media
// si añadimos 7 medidas más...hay que definir un nuevo vector!!
  int w[37];
  for (int j=0;j<37;j++){
     if (j<30) {w[j]=v[j];}
     else      {cin >> w[j];}
  }
  cout << media (w,37);
  return 0;}
```

Motivación: Uso del montón

En resumen

- La utilización de variables estáticas o automáticas para almacenar información cuyo tamaño no es conocido a priori (sólo se conoce exactamente en tiempo de ejecución) **resta generalidad al programa**
- La alternativa válida para solucionar estos problemas consiste en la posibilidad de, en tiempo de ejecución: **pedir la memoria necesaria** para almacenar la información y de **liberarla** cuando ya no sea necesaria.
- Esta memoria se reserva en el montón (Heap) y, habitualmente, se habla de **variables dinámicas** para referirse a los bloques de memoria del montón que se reservan y liberan en tiempo de ejecución.

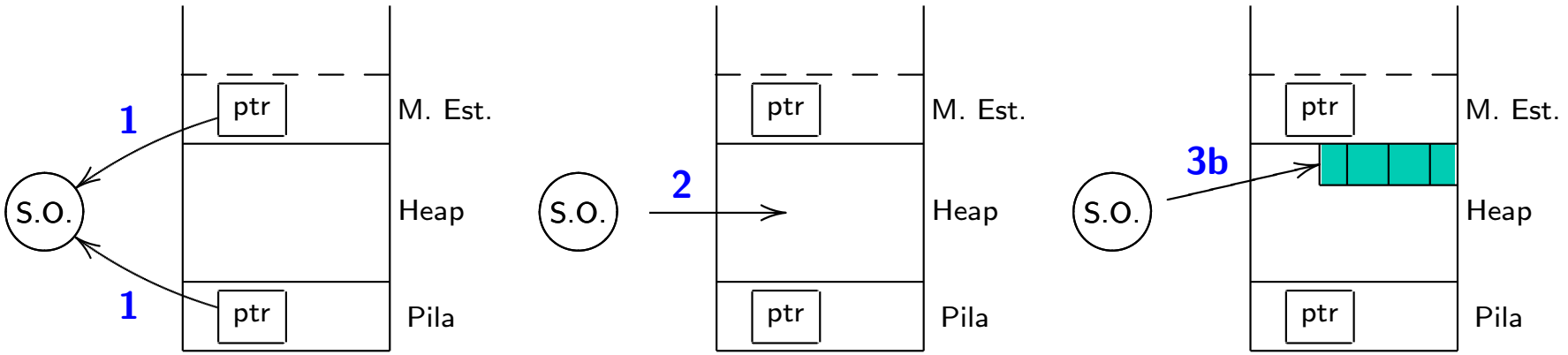
Esquema de la gestión de la memoria dinámica

Esquema

1. Petición al S.O. de memoria
2. El S.O. comprueba si hay suficiente espacio libre

3a. Si no es posible, se provoca una excepción y el programa termina

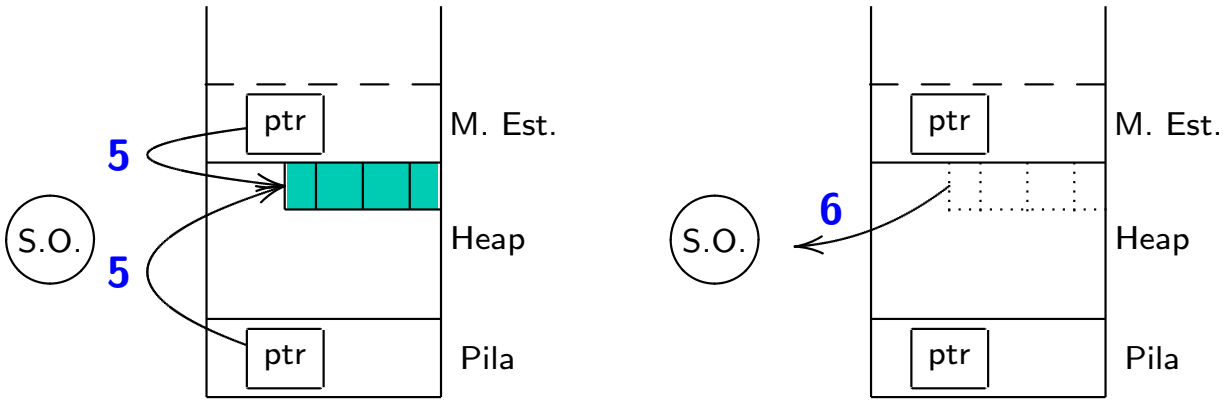
3b. Si es posible, reserva una zona memoria en el montón y la marca como memoria ocupada



Esquema de la gestión de la memoria dinámica

Esquema

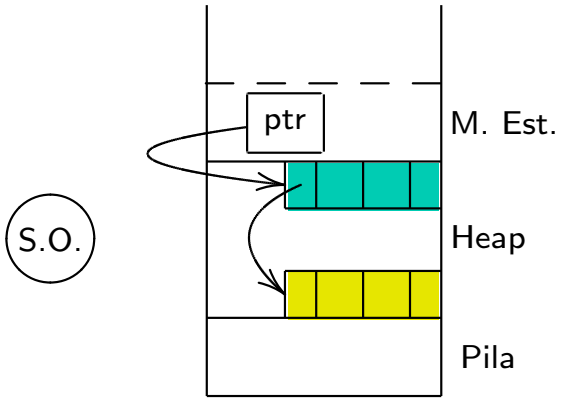
4. Devuelve la dirección de memoria
5. Esta se almacena en un puntero en la pila o en la memoria estática
6. Después de utilizarla, se libera: se informa al S.O. de que la zona de memoria vuelve a estar libre para su utilización



Esquema de la gestión de la memoria dinámica

Nota

A su vez, es posible que las nuevas variables dinámicas creadas puedan almacenar la dirección de nuevas peticiones de reserva de memoria



Reserva de memoria en el montón: operador new

La reserva de memoria se realiza con el operador `new`

Uso del operador new (nothrow)

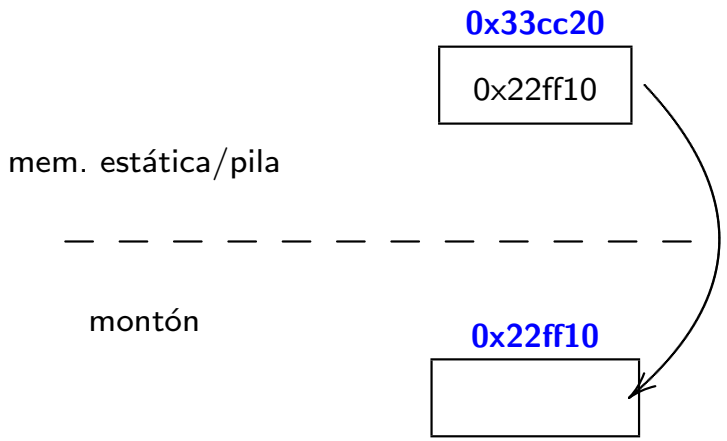
```
tipo_dato *p;
p = new (nothrow) tipo_dato ;
```

- `new` solicita reservar una zona de memoria en el montón del tamaño adecuado para almacenar un dato del tipo tipo_dato
- Es necesario declarar previamente un puntero a tipo_dato ya que `new` devuelve la dirección de memoria del principio de la zona reservada
- Si `new (nothrow)` no puede reservar espacio (por ejemplo, no hay suficiente memoria disponible), devuelve la dirección nula

Reserva de memoria en el montón: operador new

Es decir,

- Declaramos un puntero en la memoria estática
  `tipo_dato *p;`
- Que apunta una dirección de memoria del montón
  `p = new (nothrow)tipo_dato ;`



Reserva de memoria en el montón: operador new

Ejemplo

```
int main() {
  int *p;                    // declaramos un puntero a entero
  p=new (nothrow) int ;// petición de reserva de memoria
                             // en el montón,
                             //con p apuntando a esa zona

  int q=10;

  if (p!=0) // si se ha reservado memoria con exito
     *p=q;
.............
.............
  return 0;
}
```

Reserva de memoria en el montón: operador new

Para tipos de datos complejos se realiza de la misma forma

Ejemplo

```
struct DNI{
  int numero;
  char letra;
}
int main() {
  DNI *pepito; // puntero a DNI
  pepito=new (nothrow) DNI ;// reserva de memoria
  if (pepito==0) // si no se ha reservado memoria con exito
     exit(1) // salir del programa
  cin >> pepito->numero; // o bien (*pepito).numero
  cin >> pepito->letra; // o bien (*pepito).letra
.....
  return 0;}
```

Reserva de memoria en el montón: operador delete

La memoria reservada se libera usando el operador `delete`

Uso del operador delete

```
tipo_dato *p;
p = new (nothrow) tipo_dato ;
......
delete p
```

`delete` permite liberar la memoria que había sido reservada y que se encuentra referenciada por un puntero, pero:

- No "borra" la dirección memoria almacenada en el puntero
- El puntero sigue existiendo y sigue apuntando a la dirección de memoria del montón
- El valor guardado en la dirección de memoria que había sido reservada puede variar

Reserva de memoria en el montón: operador delete

Ejemplo

```
int main(){
int *p;
p = new (nothrow) int ;
*p=10;
cout << p << " " << *p << endl;
delete p;
cout << p << " " << *p;
return 0;
}
```

```
0x3d2478 10
0x3d2478 0
```

Reserva de memoria en el montón: operador delete

El puntero puede volver a utilizarse

```
int main(){
int *p;
p = new (nothrow) int ;
*p=10;
delete p;
p = new (nothrow) int ;
*p=7;
delete p;
return 0;}
```

¡¡¡CUIDADO!!!

Si cambiamos el valor del puntero antes utilizar el operador `delete`, la zona de memoria a la que apunta no se libera y no se puede volver a reservar, hemos "perdido memoria"

Reserva de memoria en el montón: operador delete

Aplicar dos veces el operador `delete` al mismo puntero, sin cambiar su valor, produce errores en tiempo de ejecución

Ejemplo

```
int *p;
p = new (nothrow) int ;
delete p;
delete p;  // liberamos la memoria dos veces
```

Solución: dar la dirección nula a los punteros

```
int *p;
p = new (nothrow) int ;
delete p;
p=0;
delete p;  // no afecta al puntero nulo
```

Reserva de memoria en el montón: operador delete

Aplicar el operador `delete` a un puntero que no apunta a una dirección del montón produce errores en tiempo de ejecución

Ejemplo

```
int v[20];
int *ptr;
ptr=v
delete ptr;  // v no es dinámico!!!
```

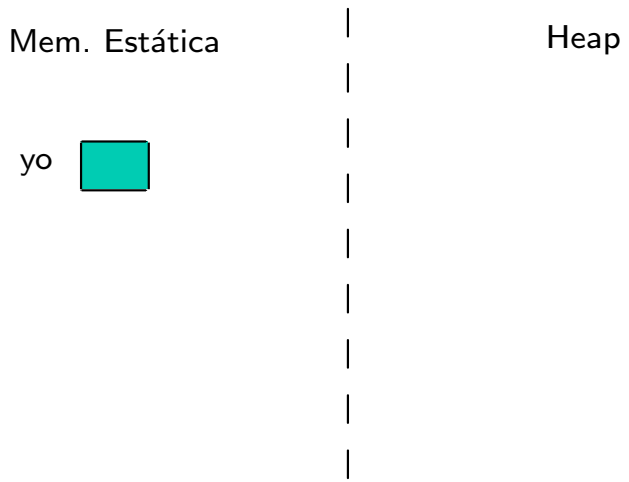
Ejemplo

Estructura de datos

```
struct Persona {
char nombre[80];
Persona *amigo;
};
```

Instrucciones

```
Persona *yo; // Declaramos un puntero a estructura Persona
```



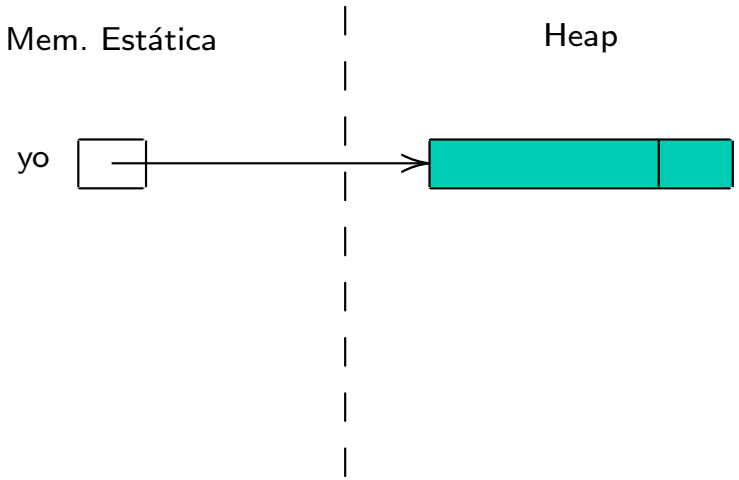
Ejemplo

Estructura de datos

```
struct Persona {
char nombre[80];
Persona *amigo;
};
```

Instrucciones

```
yo = new (nothrow) Persona ; // Reservamos memoria
```



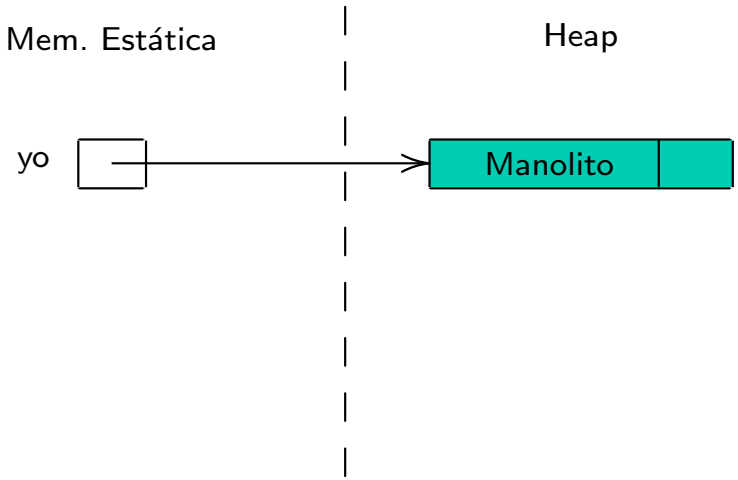
Ejemplo

Estructura de datos

```
struct Persona {
char nombre[80];
Persona *amigo;
};
```

Instrucciones

```
strcpy (yo->nombre,"Manolito"); //damos valor a yo.nombre
```

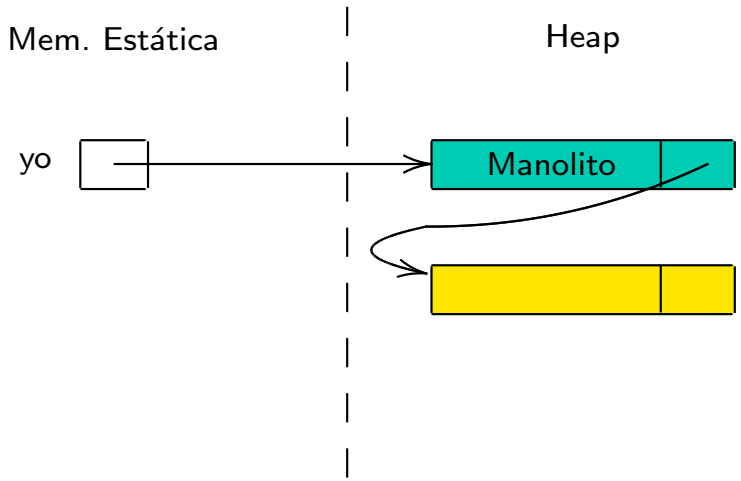


Ejemplo

Instrucciones

```
yo->amigos = new Persona;
```

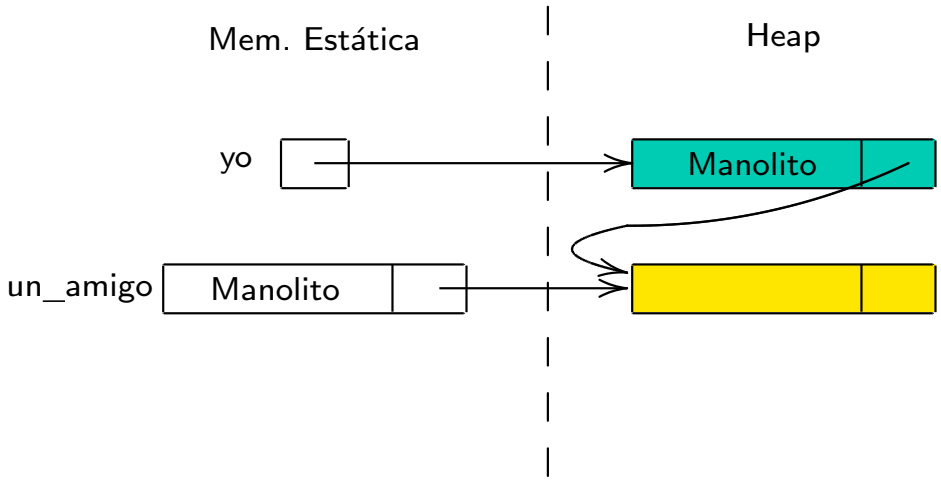
/* Reserva memoria para almacenar (en el Heap) otro dato de tipo 'Persona', que es referenciada por el campo amigos de la variable apuntada por 'yo' */



Ejemplo

Instrucciones

```
Persona un_amigo = *yo;
/* Se crea la variable estática 'un_amigo' y se realiza
una copia de la variable que es apuntada por 'yo' */
```

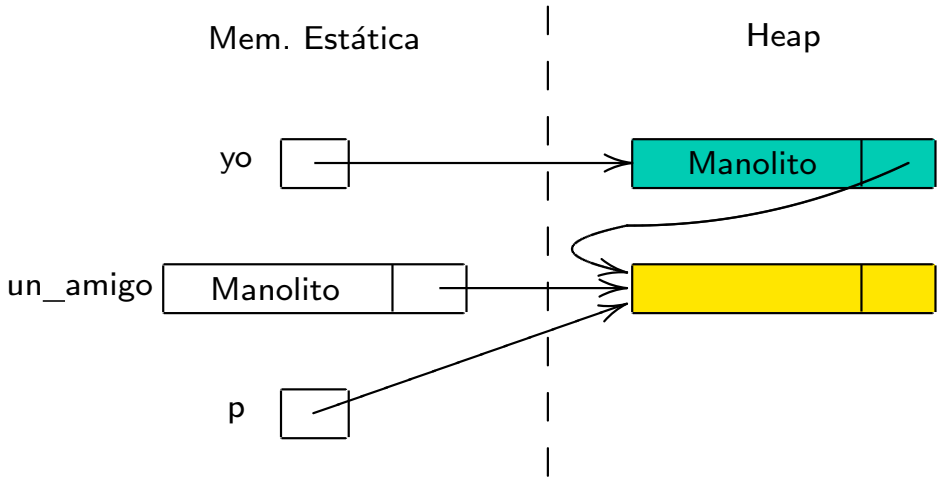


Ejemplo

Instrucciones

```
Persona *p = yo->amigos;
```

/* Se crea un puntero 'p' a 'Persona' que toma, como dirección de memoria, el valor del campo 'amigos' de la variable apuntada por 'yo' */

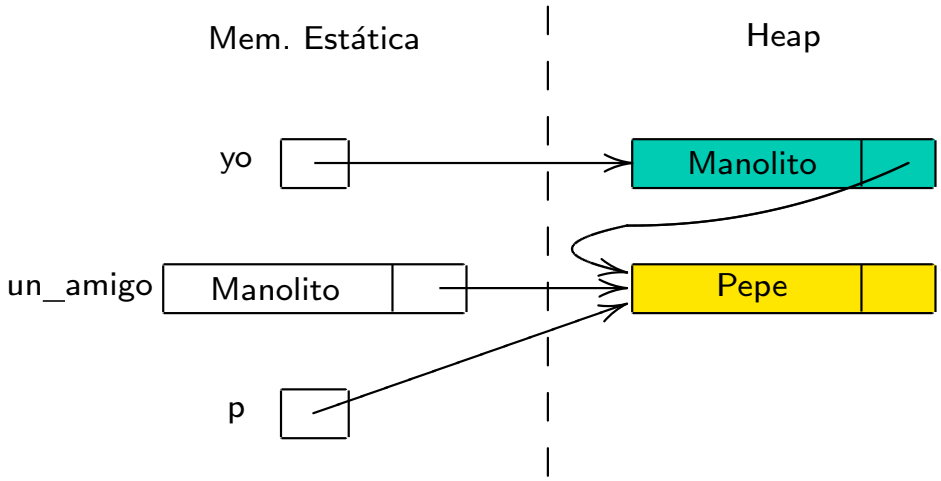


Ejemplo

Instrucciones

```
strcpy (p->nombre , "Pepe");
```

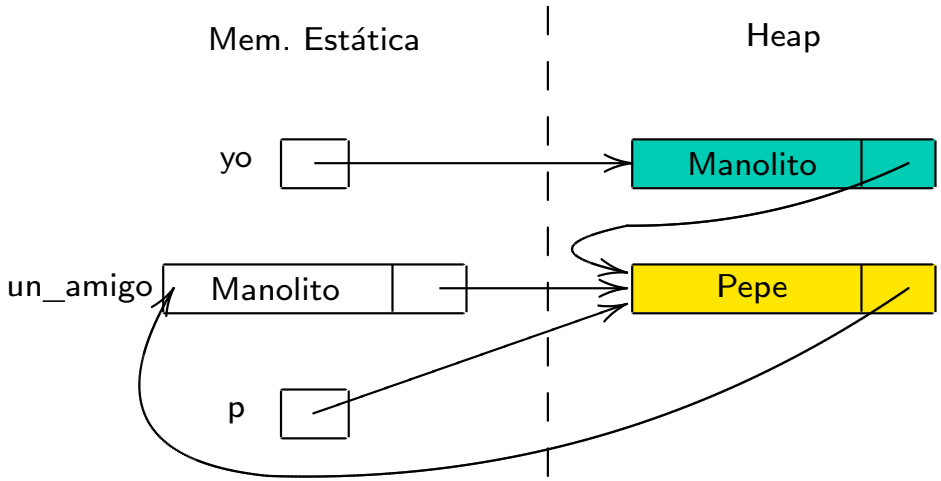
/* Usando 'p' damos valor al campo 'nombre' de la variable referenciada por el campo 'amigo' de la variable referenciada por 'yo' */



Ejemplo

Instrucciones

```
p->amigo = & un_amigo ;
/* Es posible hacer que una variable dinámica apunte a una
variable automática o estática usando el operador de dirección */
```

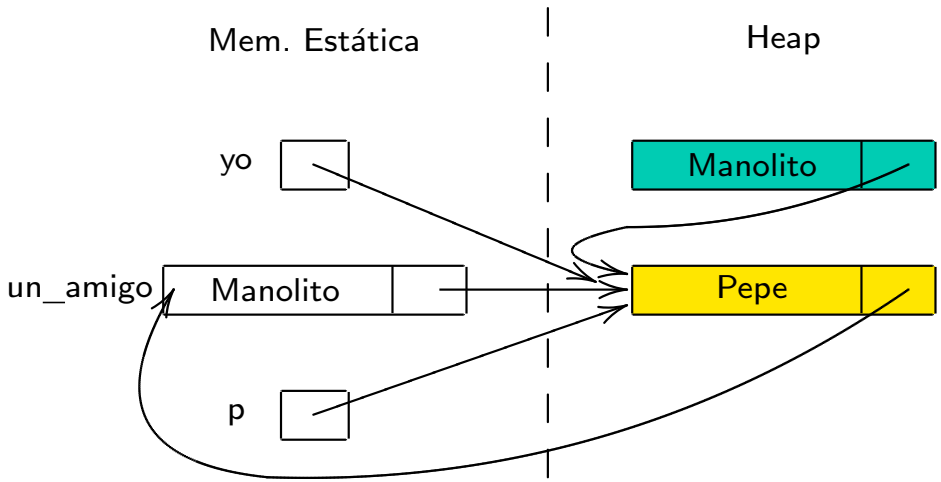


Ejemplo

CUIDADO

Si hacemos `yo = p ;` perderemos la dirección de un objeto referenciado y no podremos liberar esa zona de la memoria. Es conveniente:

- Liberar antes la memoria reservada, o bien
- Tener otro puntero que referencie dicho objeto

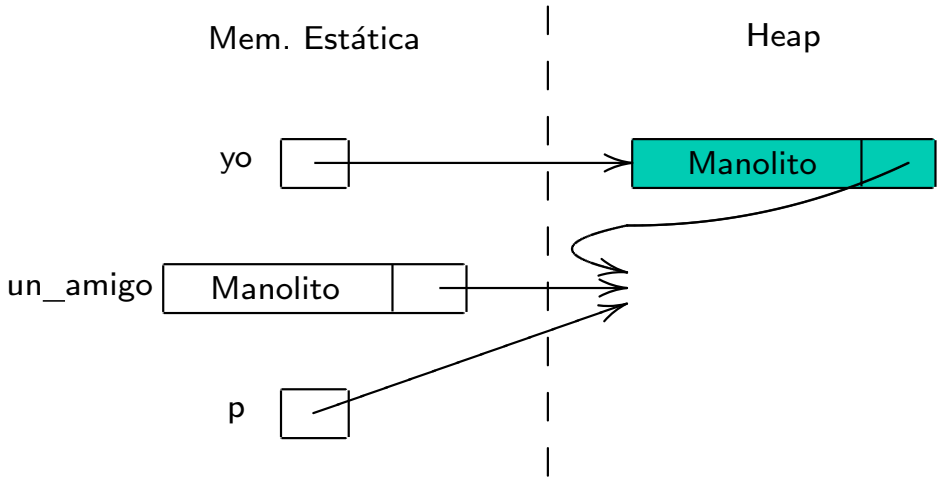


Ejemplo

Liberemos la memoria de manera correcta

Instrucciones

```
delete un_amigo.amigos;
/* Liberamos la memoria cuya dirección de memoria
está almacenada en el campo 'amigos' de la variable 'un_amigo'. O
bien, delete yo->amigos */
```

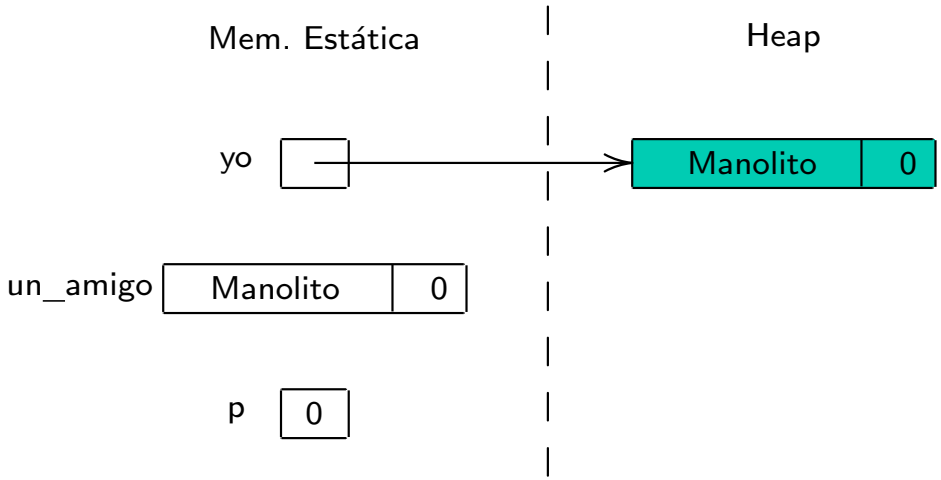


Ejemplo

Liberemos la memoria de manera correcta

Instrucciones

```
yo->amigos = un_amigo.amigos = p = 0;
/* Damos la dirección nula a todos los punteros que apuntaban a
la memoria liberada, para no realizar operaciones con dicha
memoria o utilizar delete con ellos */
```

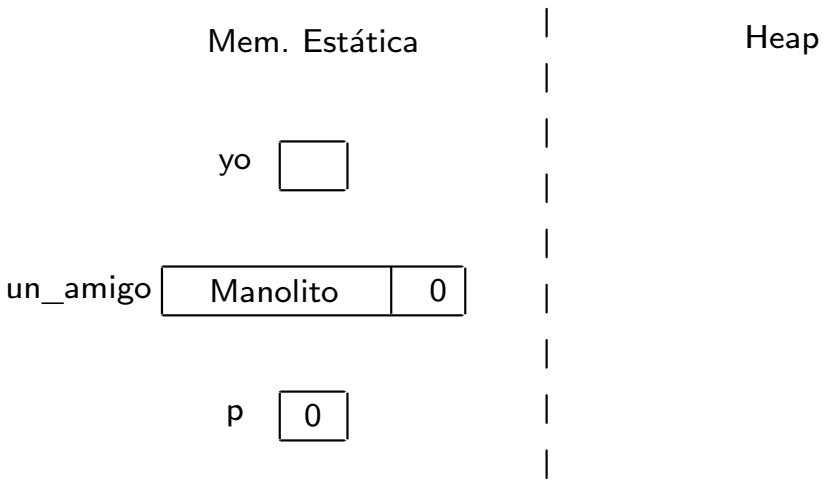


Ejemplo

Liberemos la memoria de manera correcta

Instrucciones

```
delete yo;
/* liberamos la memoria a la apunta 'yo' */
```

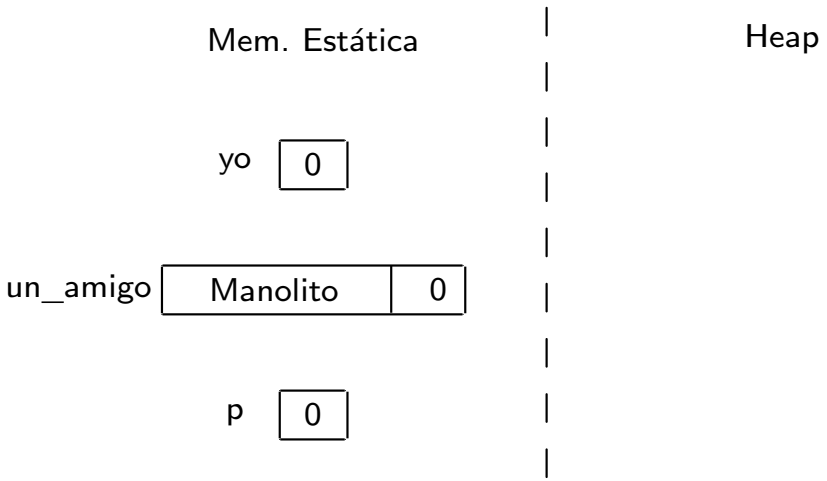


Ejemplo

Liberemos la memoria de manera correcta

Instrucciones

```
yo = 0;
/* Damos la dirección nula a 'yo' por si nos equivocamos */
```



Reserva de memoria en el montón: operador delete

Por tanto, recordemos la metodología de la memoria dinámica

Metodología

1. Reservar memoria con `new`
2. Controlar un posible fallo de memoria
3. Utilizar la memoria reservada
4. Liberar la memoria reservada con `delete`

Consejo

La memoria siempre debe liberarse (correctamente) para no producir "perdidas de memoria"

Vectores dinámicos: operador new []

La reserva de memoria para almacenar un vector se realiza con el operador `new []`

Uso del operador new (nothrow) []

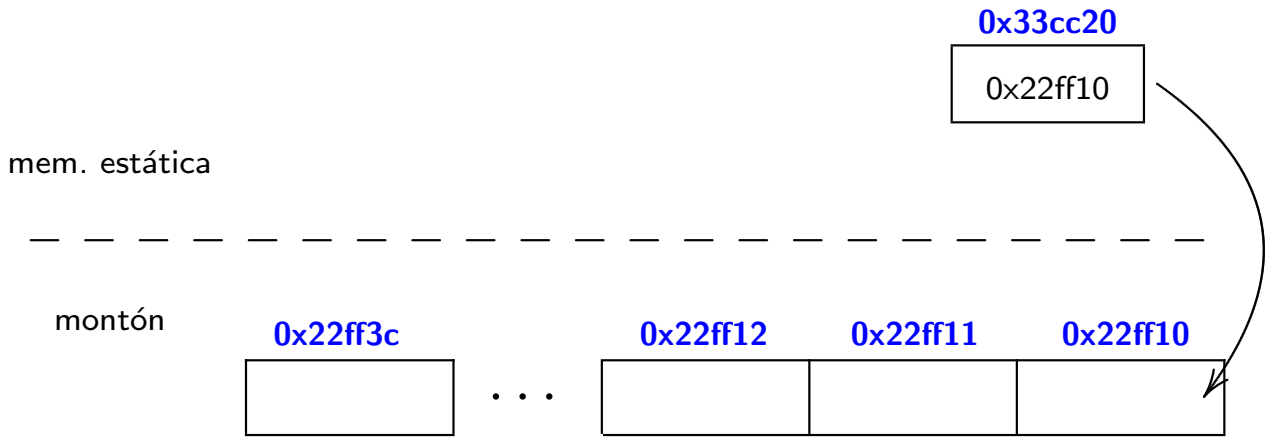
```
tipo_dato *p;
p = new (nothrow) tipo_dato [numero];
```

- `new []` solicita reservar una zona de memoria en el montón para almacenar `numero` datos de tipo tipo_dato
- Se necesita declarar previamente un puntero ya que `new []` devuelve la dirección de memoria inicial
- Si `new (nothrow) []` no puede hacer la reserva de memoria devuelve la dirección nula

Vectores dinámicos: operador new []

Es decir,

- Declaramos un puntero en la memoria estática
  tipo_dato *p;
- Que apunta a la primera dirección de memoria del vector reservado en el montón
  p = new (nothrow) tipo_dato [numero];



Vectores dinámicos: operador delete []

La memoria reservada se libera usando el operador `delete []`

Uso del operador delete []

```
tipo_dato *p;
p = new (nothrow) tipo_dato [numero];
......
delete [] p;
```

`delete []` libera (pone como disponible) la memoria previamente reservada por `new []` y que está referenciada por el puntero `p`, pero:

- El puntero sigue existiendo y sigue apuntando a la dirección de memoria inicial
- Asigna el valor 0 a la dirección de memoria inicial

Vectores dinámicos

Resumen

Utilizando vectores dinámicos conseguimos:

- crear un vector que tenga justo el tamaño necesario, conociendo ese tamaño en tiempo de compilación
- crearlo cuando lo necesitemos y destruirlo cuando deje de ser útil
- por tanto, no desperdiciar una parte de la memoria, que podría sernos útil a posteriori
- programas más generales

Vectores dinámicos

Ejemplo

```
int main() {
  int *v, n;
  cout << "longitud del vector: ";
  cin >> n;

  v=new (nothrow) int [n]; // 1. reserva de memoria

  if (v!=0){ exit (1);} // 2. comprobamos si se reservó

  for (int i=0; i<n ; i++) //
     cin >> v[i];          // 3. utilizamos el vector reservado
  for (int j=0; j<n ; j++) //
     cout << v[i] << " "; //

  delete [] v;  // 4. liberamos la memoria

  return 0;}
```

Vectores dinámicos

Ejemplo

```
int main() {
  int *v, n;
  cout << "longitud del vector: ";
  cin >> n;

  v=new (nothrow) int [n]; // 1. reserva de memoria

  if (v!=0){ exit (1);} // 2. comprobamos si se reservó

  for (int i=0; i<n ; i++) //
     cin >> v[i];          // 3. utilizamos el vector reservado
  for (int j=0; j<n ; j++) //
     cout << v[i] << " "; //

  delete [] v;  // 4. liberamos la memoria

  return 0;}
```

Vectores dinámicos

Función que amplia el espacio ocupado por un vector

```
void Ampliar (int &v[], int elementos, int ampliacion){

  int *v_ampliado;
  v_ampliado=new (nothrow) int [elementos+ampliacion];

  if (!v_ampliado) {
     cerr << "Error de memoria\n";
     exit (1);
  }

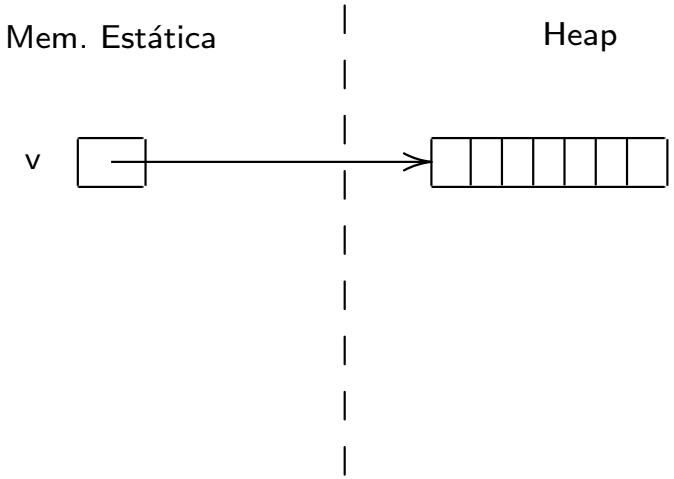
  for (int i= 0; i<elementos; i++)
     v_ampliado[i] = v[i];

  delete [] v;// liberar memoria antes de modificar v
  v = v_ampliado;// modificar v, por eso pasa por referencia

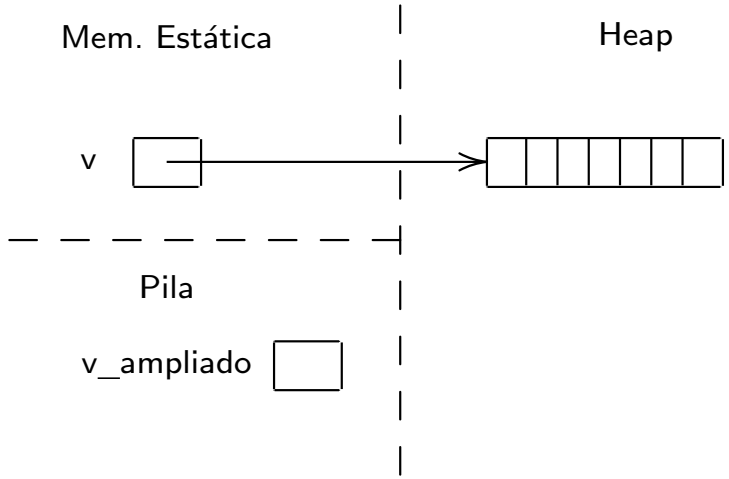
}
```

Vectores dinámicos

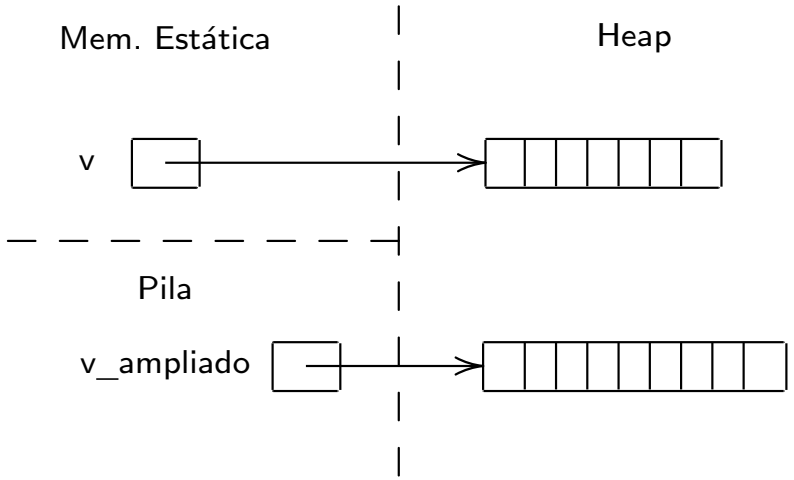
Situación original



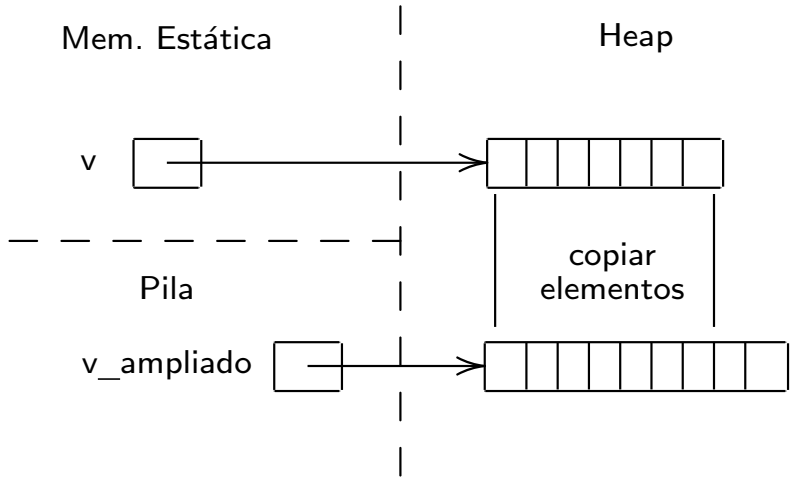
Declarar puntero v_ampliado



Asignar memoria a v_ampliado

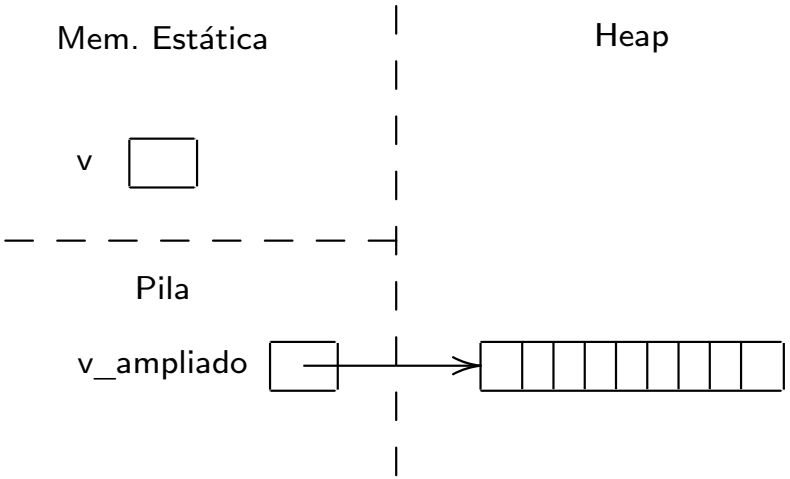


Copiar elementos del vector

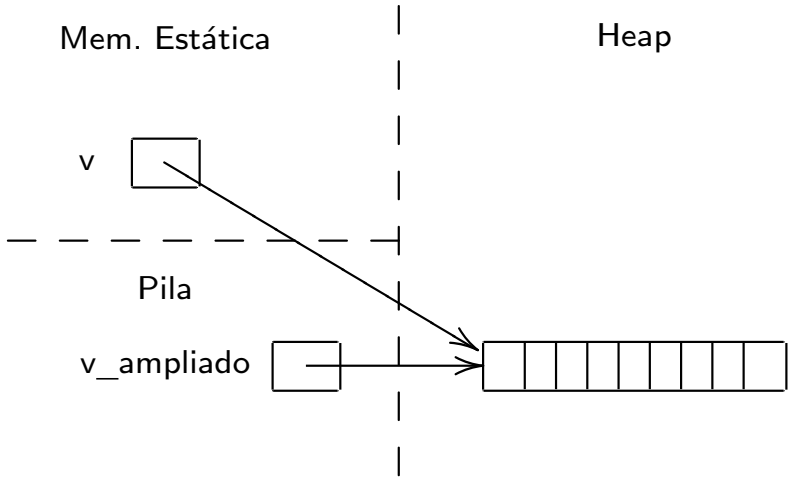


Vectores dinámicos

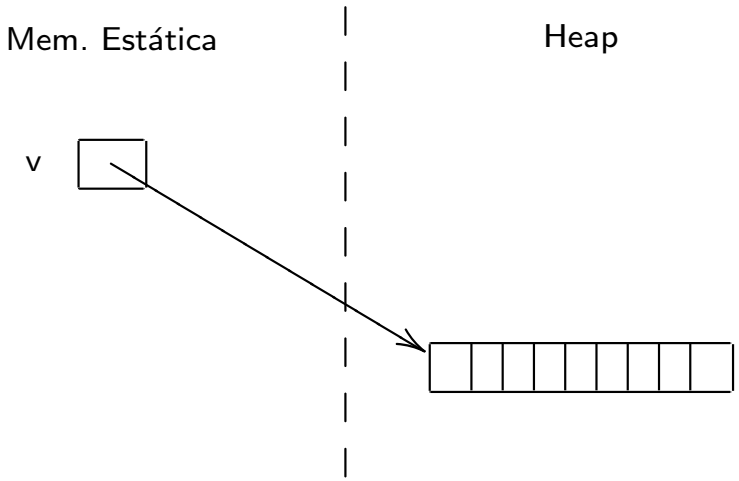
Liberar memoria de v



v=v_ampliado



Fin de funcion Ampliar



Vectores dinámicos

Función que devuelve un vector copia de otro

```
int * copiar_vector (int v[], int n){

  int *copia;
  copia = new (nothrow) int [];

  if (copia!=0){
     for (int i=0; i<n; i++)
        copia[i]=v[i];
  }

  return copia;
}
```

Función main

```
int main(){

  int *copia;
  int vector[30]={0};

  copia=copiar_vector (vector,30);

  if (copia==0)
     exit (1);

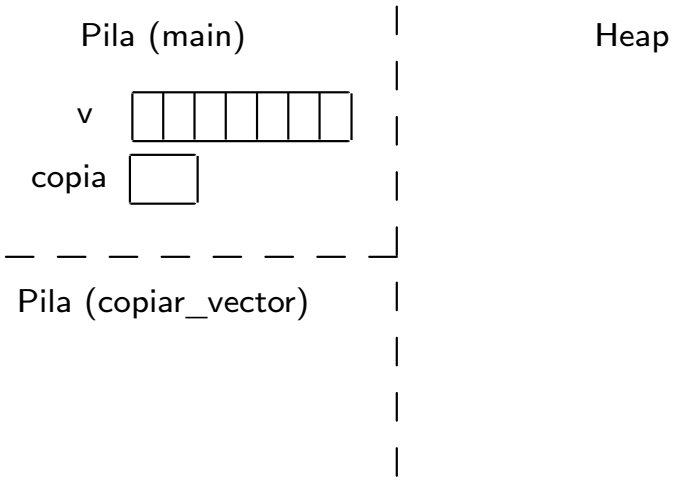
// uso del vector copia

  delete copia;// liberamos la memoria reservada por la función copiar_vector

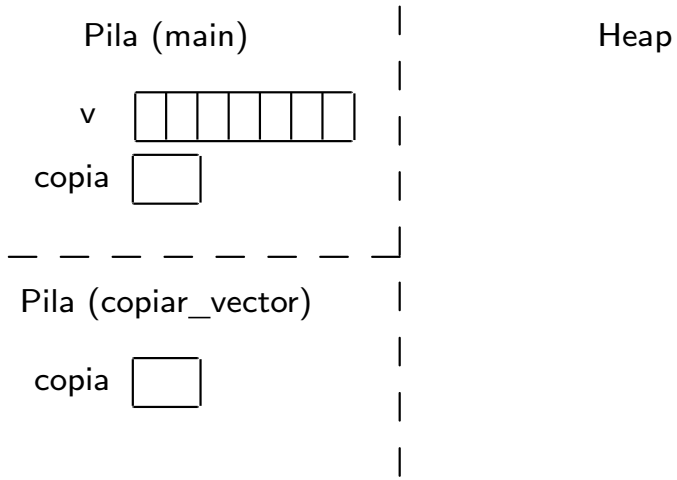
  return 0;
}
```

Vectores dinámicos

main: **int ∗copia, v[30]**

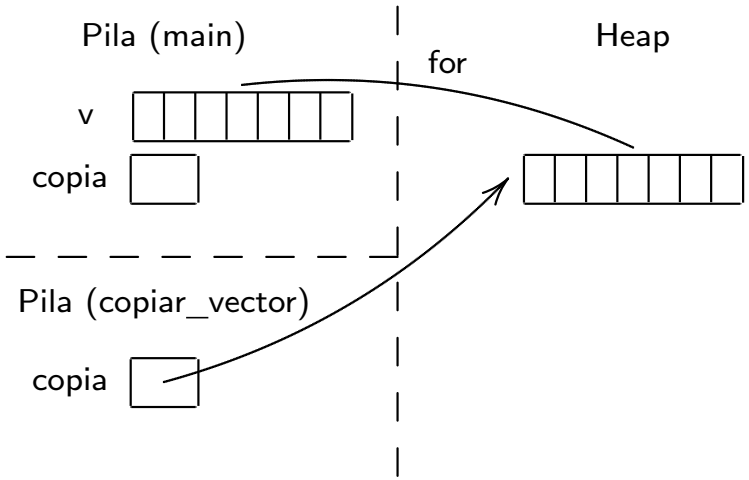


copiar_vector: **int ∗copia**



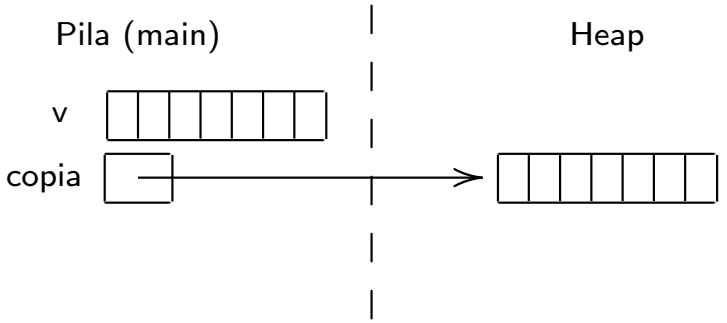
copiar_vector: **copia = new (nothrow) int [n]**

copiar_vector: bucle for

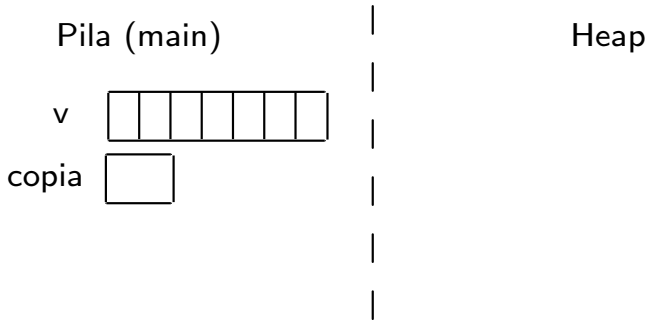


Vectores dinámicos

main: **copiar=copiar_vector (v,30)**



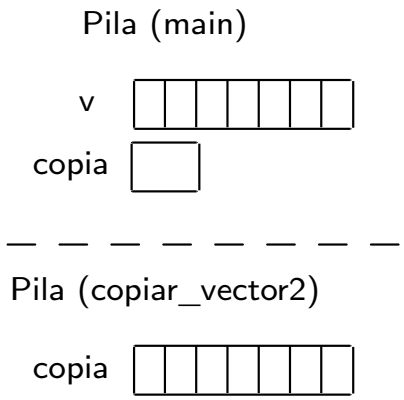
main: **delete copia**

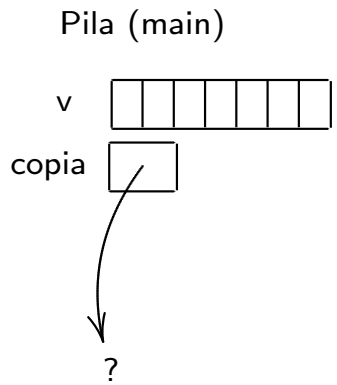


Vectores dinámicos

CUIDADO: Error muy común

```
int * copia_vector2( int v[], int n){
  int copia[100]; // o cualquier otro valor mayor que n
  for (int i= 0; i<n; i++)
     copia[i] = v[i];
  return (copia);
}
// copia es una variable local,
// no puede ser usada fuera del ámbito de la función
```





Matrices dinámicas

Motivación

Necesitamos gestionar matrices de dos dimensiones de forma dinámica (en tiempo de ejecución) cuyo tamaño, filas y columnas, sea exactamente el que requiera el problema a resolver

Problema

No existe un comando específico para crear una matriz. Algo del estilo `new int [filas][columnas]`

Es necesario usar vectores dinámicos. Dos formas de resolver el problema:

- Datos guardados en filas independientes
- Datos guardados en una única fila

Matrices dinámicas: filas independientes

Se necesita reservar memoria para una matriz filas x columnas

Solución 1

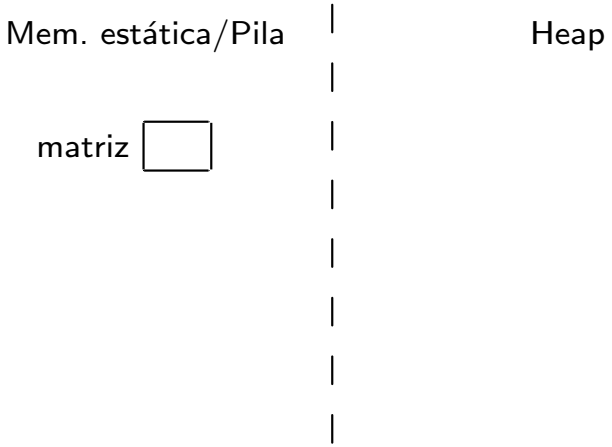
1. Declarar un puntero a puntero
   ```
   int ** matriz;
   ```
2. Reserva memoria para un vector de punteros de filas elementos
   ```
   matriz = new (nothrow) int * [filas];
   ```
3. Reservar, para cada fila, un vector de columnas elementos
   ```
   for ( int i=0 ; i<filas ; i++)
       matriz[i]=new (nothrow) int [columnas]
   ```

Nota

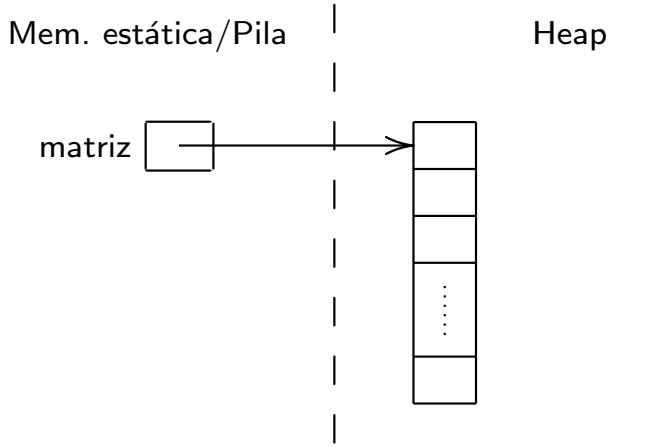
En cada paso de reserva hay que verificar si existe memoria suficiente

Matrices dinámicas: filas independientes

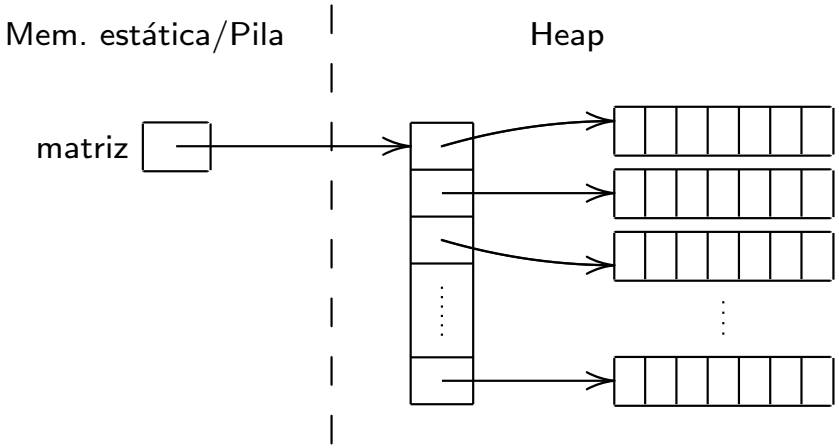
int ** matriz



matriz=new int * [filas]



bucle: **matriz[i]=new int [columnas]**



Matrices dinámicas: filas independientes

Función de reserva (1)

```
int ** CrearMatriz1 ( int filas, int columnas){

 bool error = false;
 int ** matriz; // puntero a puntero a entero, para almacenar vector de punteros

 matriz = new (nothrow) int * [filas];//reserva de vector de punteros

 if (!matriz) // en caso de no poder reservar memoria...
    cerr << "Error en reserva" << endl; // mensaje de error
 else {  // si se ha creado el vector de punteros...
    for (int f=0; (f<fils) && !error; f++) {
       matriz[f] = new (nothrow) int [columnas];// cada matriz[f] apunta a vector de enteros
       if (!matriz[f]) { // si no es posible reservar memoria...
          cerr << "Error en reserva (2)" << endl; //mensaje de error
          error = true;// detenemos el bucle for
          for (int i=f-1; i>=0; i--)
             delete [] matriz[i]; // hay que liberar la memoria ya reservada
          delete [] matriz; // liberamos el vector de punteros
          matriz = 0; // puntero nulo, para no equivocarnos
       }
    }
 }

 return (matriz);  // devuelve el puntero a puntero
}
```

Matrices dinámicas: filas independientes

Función de liberación de memoria (1)

```
void LiberarMatriz1(int ** matriz,int filas,int columnas){

 for (int f=0 ; f<filas ; f++)
    delete [] matriz[f];// liberamos cada vector de enteros

 delete [] matriz; // liberamos el vector de punteros
}
```

¡¡¡CUIDADO!!!

La memoria debe liberarse en el orden correcto. Si liberamos antes el vector de punteros, perdemos la referencia a los vectores de enteros y no podrán liberarse

Matrices dinámicas: en una sóla fila

Se necesita reservar memoria para una matriz filas x columnas

Solución 2

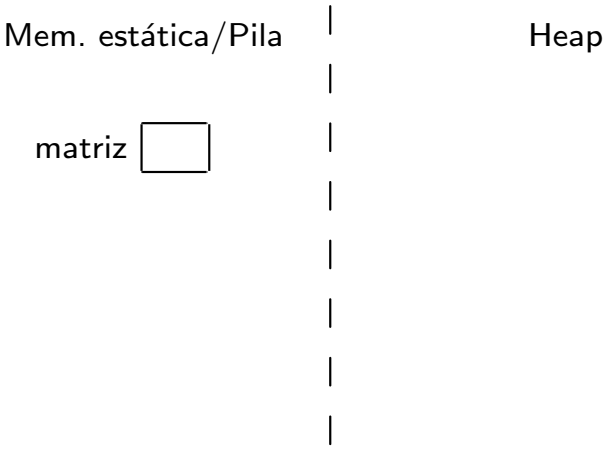
1. Declarar un puntero a puntero
   ```
   int ** matriz;
   ```
2. Reserva memoria para un vector de punteros de filas elementos
   ```
   matriz = new (nothrow) int * [filas];
   ```
3. Reservar un único vector de filas*columnas elementos
   ```
   matriz[0] = new (nothrow) int [filas*columnas];
   ```
4. Acceder a partes del vector desde los punteros
   ```
   for( int i=1 ; i<filas ; i++)
      matriz[i] = matriz[i-1] + columnas;
   ```

Nota

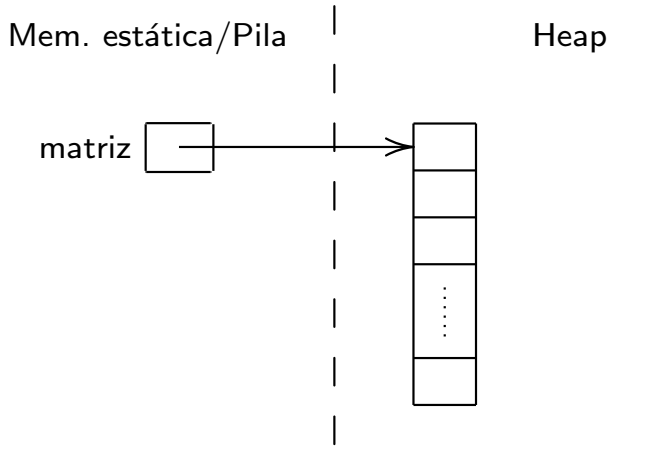
En cada reserva hay que verificar si existe memoria suficiente

Matrices dinámicas: en una sóla fila

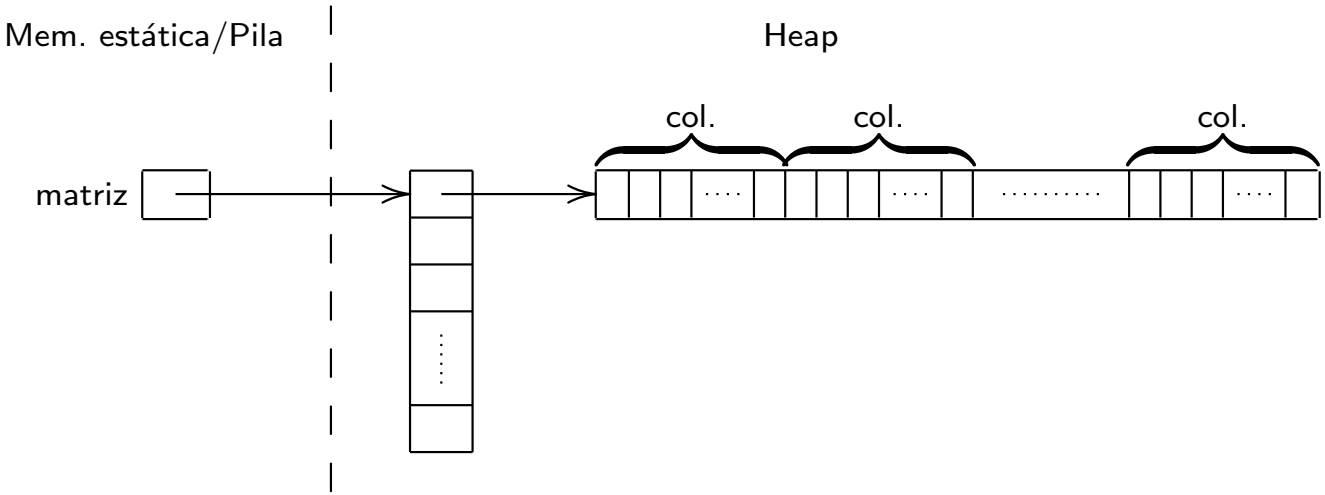
int ** matriz



matriz=new int * [filas]

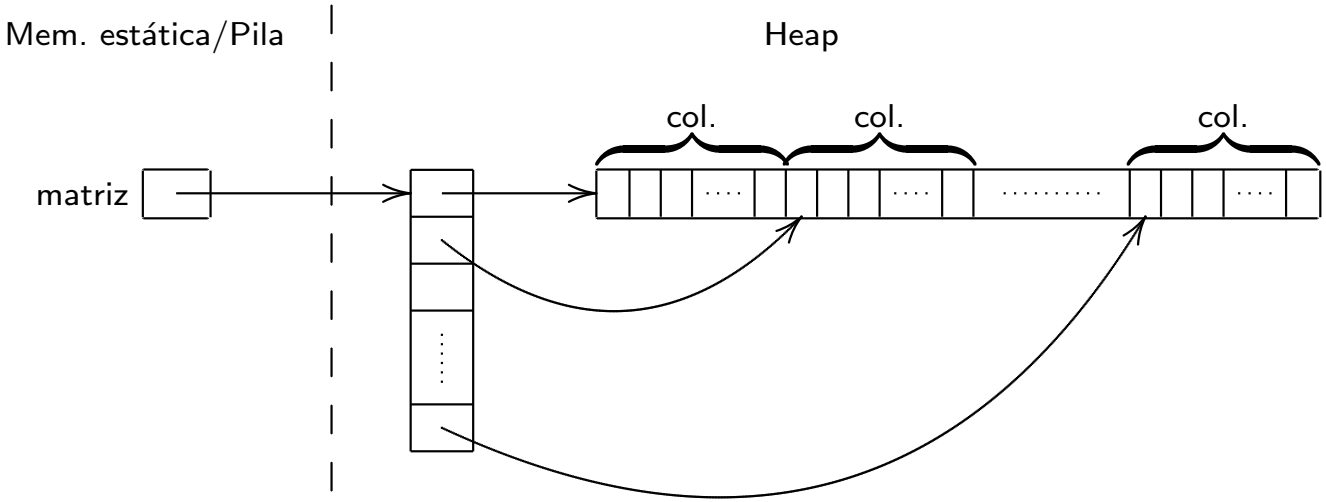


bucle: **matriz[0] = new (nothrow) int [filas*columnas]**



Matrices dinámicas: en una sóla fila

bucle: **matriz[i] = matriz[i-1] + columnas**



Matrices dinámicas: en una sóla fila

Función de reserva (2)

```
int ** CrearMatriz2 ( int filas, int columnas){

 int ** matriz; // puntero a puntero a entero, para almacenar vector de punteros

 matriz = new (nothrow) int * [filas];// reserva de vector de punteros

 if (!matriz) // en caso de no poder reservar memoria...
    cerr << "Error en reserva" << endl; // mensaje de error

 else {  // si se ha creado el vector de punteros...
    matriz[0] = new (nothrow) int [filas*columnas];// reservar un vector de filas*columnas enteros

    if (!matriz[0]) { // si no es posible reservar memoria...
       cerr << "Error en reserva (2)" << endl; //mensaje de error
       delete [] matriz; // liberamos el vector de punteros
       matriz = 0; // puntero nulo, para no equivocarnos
    else
       for ( int i=1 ; i < filas ; i++)
          matriz[i]=matriz[i-1]+ columnas; // asignamos a cada puntero un elemento del vector
 }

 return (matriz);  // devuelve el puntero a puntero
}
```

Matrices dinámicas: filas independientes

Función de liberación de memoria (2)

```
void LiberarMatriz1(int ** matriz,int filas,int columnas){
  delete [] matriz[0];// liberamos el vector de enteros
  delete [] matriz; // liberamos el vector de punteros
}
```

¡¡¡CUIDADO!!!

La memoria debe liberarse en el orden correcto. Si liberamos antes el vector de punteros, perdemos la referencia al vector de enteros y no podrá liberarse

Matrices dinámicas: Ejemplo

Ejemplo

```
#include <iostream>
#include <iomanip>
#include <new>
using namespace std;

void LeeDimensiones (int &num_filas, int &num_cols);
int ** CrearMatriz1 (int filas, int cols);
int ** CrearMatriz2 (int filas, int cols);
void PintarMatriz (int ** matriz, int filas, int cols);
void LiberarMatriz1 (int ** matriz, int filas, int cols);
void LiberarMatriz2 (int ** matriz, int filas, int cols);
```

Matrices dinámicas: Ejemplo

Función main

```
int main(){

 int ** m1;
 int ** m2; // m1 y m2 serán matrices dinámicas

 int filas, cols; // variables para el número de filas y columnas de las matrices

 LeerDimensiones (filas, cols); // leer el número de filas y columnas

 cout << "Creando Matriz 1 ("<< filas << "X"<< cols << ")" << endl; // Crear una matriz dinámica
 m1 = CrearMatriz1 (filas, cols);

//******* en caso de que no se pueda reservar memoria ********//
 if (!m1) {
    cerr << "No se pudo alojar matriz." << endl;
    exit (1);
 }
//*********************************************************//

 cout << "Creando Matriz 2 ("<< filas << "X"<< cols << ")" << endl;// Crear otra matriz dinámica
 m2 = CrearMatriz2 (filas, cols);

//******* en caso de que no se pueda reservar memoria ********//
 if (!m2) {
    cerr << "No se pudo alojar matriz." << endl;
    exit (1);
 }
//*********************************************************//
```

Matrices dinámicas: Ejemplo

Función main

```
  cout << endl << "Rellenando matrices" << endl;// Rellenar las matrices (observar el acceso por indices)
  for (f=0; f<filas; f++)
     for (c=0; c<cols; c++) {
        m1[f][c] = ((f+1)*10)+c+1;
        m2[f][c] = ((f+1)*10)+c+1;
     }

  cout << endl << "Matriz 1:" << endl; // Mostrar las matrices
  PintarMatriz (m1, filas, cols);
  cout << endl << "Matriz 2:" << endl;
  PintarMatriz (m2, filas, cols);

  cout << endl << "Liberando matriz 1" << endl; // liberar la memoria ocupada
  LiberarMatriz1 (m1, filas, cols);
  cout << "Liberando matriz 2" << endl;
  LiberarMatriz2 (m2, filas, cols);

  return 0;
}
```

Matrices dinámicas: Ejemplo

Función LeerDimensiones

```
void LeerDimensiones (int &num_filas, int &num_cols) {
  cout << "Numero de filas : ";
  cin >> num_filas;
  cout << "Numero de columnas : ";
  cin >> num_cols;
}
```

Función PintarMatriz

```
void PintarMatriz (int ** matriz, int filas, int cols) {
  for (int f=0 ; f<filas ; f++) {
     for (int c=0; c<cols ; c++)
        cout << setw(4) << matriz[f][c];
     cout << endl;
  }
}
```

Listas Enlazadas: Motivación

Reservar y liberar un vector con memoria dinámica es una operación lenta y poco eficiente.

Si el vector cambia de tamaño con frecuencia dentro del programa, se necesitarán múltiples reservas nuevas de memoria, operaciones de copia del vector, y liberación de la memoria diñamica anticuada.

Solución: Reservar/liberar componente a componente de forma independiente, sin necesidad de alterar el resto del vector ni sus componentes.

C++ no permite alterar vectores de esta forma. Para ello, se utilizan listas enlazadas.

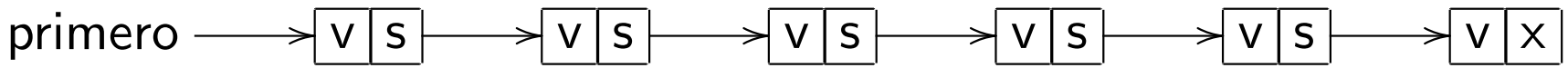
Listas Enlazadas: Estructura de una celda

La estructura de una celda, en una lista enlazada, debe contener su valor y, al menos, una referencia (puntero) a su elemento siguiente.

Ejemplo de estructura celda

```
struct celdaEntero {
  int v; // Valor de la componente
  celda *s; // Puntero al siguiente elemento (tipo celda)
}
```

Ejemplo de lista enlazada:



Se reconoce el final de la lista porque el campo **s** del último elemento apunta a la dirección nula.

Listas Enlazadas: Creación y liberación de celdas

Función para crear una celda

```
celdaEntero * CrearCelda(){
  celdaEntero  *aux;
  aux= new (nothrow) celdaEntero ;
  if (aux == 0)  return  0;
  aux->s= 0;//hacemos que la celda no tenga siguiente
  return  aux;
}
```

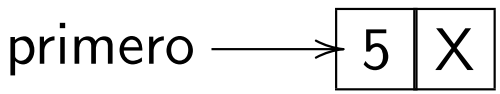
Función para liberar una celda

```
void LiberarCelda(celdaEntero * &c){
  delete c;
  c= 0;
}
```

Listas Enlazadas: Creación y liberación de celdas

Ejemplo de uso

```
int main(){
  celdaEntero  *primero;
  primero= CrearCelda();
  primero->v= 5;
  LiberarCelda(primero);
  return  0;
}
```

Resultado: 

Listas Enlazadas: Inserción de celdas al final

Función para insertar una celda al final

```
void InsertarFinal(celdaEntero *primero, int elemento){
  celdaEntero *aux= primero, *nuevo;

  while (aux->s != 0)  // Vamos al último elemento, pasando
      aux= aux->s; // de una celda a su celda siguiente

  nuevo= CrearCelda();
  nuevo->v= elemento;
  aux->s= nuevo;
}
```

Ejemplo: InsertarFinal(primero, 7);

Listas Enlazadas: Inserción de celdas al final

Función para insertar una celda al final

```
void InsertarFinal(celdaEntero *primero, int elemento){
  celdaEntero *aux= primero, *nuevo;

  while (aux->s != 0)  // Vamos al último elemento, pasando
      aux= aux->s; // de una celda a su celda siguiente

  nuevo= CrearCelda();
  nuevo->v= elemento;
  aux->s= nuevo;
}
```

Ejemplo: InsertarFinal(primero, 1);

Listas Enlazadas: Acceder a una celda

Función para acceder a la celda en una posición de la lista

```
celdaEntero * DevolverCelda(celdaEntero *primero, int posicion){
  celdaEntero *aux= primero;
  int i ;

  for (i=0; i<posicion; i++)
       aux= aux->s; // Avanzamos de celda
  return  aux;
}
```

Listas Enlazadas: contar elementos en la lista

Función para contar los elementos de la lista

```
int  NumeroElementos(celdaEntero  *primero){
  celdaEntero *aux= primero;
  int i= 1 ;

  while (aux->s != 0) {
       aux= aux->s; // Avanzamos de celda
       i++; // Aumentamos el contador de celdas
  }
  return  i;
}
```

Listas Enlazadas: Liberar una lista

Función para liberar todos los elementos de la lista

```
void LiberarLista(celdaEntero  * &primero){
  celdaEntero *aux, *sigaux;

  // sigaux será el elemento siguiente a aux
  // Primero calculamos sigaux, para no perder el elemento siguiente
  // Después liberamos aux
  // Por último, pasamos aux a la celda siguiente
  aux= primero;
  while (aux != 0) {
      sigaux= aux->s;
      LiberarCelda(aux);
      aux= sigaux;
  }
  primero= 0;
}
```

Listas Enlazadas: Ejemplo completo

Ejemplo de uso de listas

```
int main(){
  int i;
  celdaEntero  *primero;

  primero= CrearCelda();
  primero->v= 5;
  InsertarFinal(primero, 7);
  InsertarFinal(primero, 1);
  InsertarFinal(primero, 8);

  for (i= 0; i<NumeroElementos(primero); i++)
      cout<<"Elemento i:"<<(DevolverCelda(primero, i))->v<<endl;

  LiberarLista(primero);
  return  0;
}
```

Ejemplo: Diccionario (I)

Una entrada en un diccionario puede verse como un par *(palabra, descripción)*, ambos cadenas de caracteres. No obstante, no conocemos a priori la longitud máxima ni de la palabra ni de la descripción.

Podemos elaborar una estructura donde ambas componentes se reserven con memoria dinámica en tiempo de ejecución.

Modelo de una entrada en un diccionario

```
struct entrada {
  char *palabra;
  char *descripcion;
}
```

Ejemplo: Diccionario (II)

Función para reservar una entrada

```
entrada * ReservaEntrada(char *palabra, char *descripcion){
  entrada * ent;

// Reservamos para la entrada
ent= new (nothrow) entrada ;

// Reservamos para la palabra
ent->palabra= new (nothrow) char [strlen(palabra)+1];
strcpy(ent->palabra, palabra);

// Reservamos para la descripción
ent->descripcion= new (nothrow) char [strlen(descripcion)+1];
strcpy(ent->descripcion, descripcion);

  return  ent;
}
```

Ejemplo: Diccionario (III)

Función para liberar una entrada

```
void LiberaEntrada(entrada * &ent){

  // liberamos la palabra
  delete [ ] ent->palabra;

  // Liberamos la descripción
  delete [ ] ent->descripcion;

  // Liberamos la entrada
  delete ent;
}
```

Ejemplo: Diccionario (IV)

Un diccionario puede verse como una lista de entradas ordenadas alfabéticamente por el campo palabra. Además, puede tener una editorial, año de edición, ISBN, etc.

Estructura que modela un diccionario

```
struct diccionario {
  char *editorial, *isbn;
  int anio;
  celdaEntrada *palabras; // Lista de palabras del diccionario
}
```

Estructura que modela una celda de la lista de entradas

```
struct celdaEntrada {
  entrada *ent; // Entrada de la celda
  celdaEntrada *sig; // Puntero a la celda siguiente
}
```

Ejemplo: Diccionario (V)

Reserva memoria para una celda de la lista

```
celdaEntrada * CrearCelda(char *palabra, char *descripcion){
  celdaEntrada *celda;
  celda= new (nothrow) celdaEntrada ;
  celda->ent= CrearEntrada(palabra, descripcion);
  celda->sig= 0;
  return  celda;
}
```

Libera memoria reservada para una celda

```
void LiberarCelda(celdaEntrada * &celda){

  LiberaEntrada(celda->ent);
  delete [ ] celda;
  celda= 0;
}
```

Ejemplo: Diccionario (VI)

Reserva memoria para un diccionario

```
diccionario * CrearDiccionario(char *editorial, char *isbn, int anio){
  diccionario  * dic;

  dic= new (nothrow) diccionario ;
  dic->editorial= new (nothrow) char [strlen(editorial)+1];
  strcpy(dic->editorial, editorial);
  dic->isbn= new (nothrow) char [strlen(isbn)+1];
  strcpy(dic->isbn, isbn);
  dic->anio= anio;
  dic->palabras= 0; // Suponemos diccionario vacio
  return  dic;
}
```

Ejemplo: Diccionario (VII)

Libera memoria reservada para un diccionario

```
void LiberarDiccionario(diccionario * &dic){

  LiberaLista(dic->palabras);
  delete [ ] dic->editorial;
  delete [ ] dic->isbn;
  delete dic;
  dic= 0;
}
```